МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №3

по дисциплине «Интеллектуальные методы анализа данных»

Тема: Метод нечеткой кластеризации с-средних

Студент гр. 3384 _____ Горский К.Д.

Преподаватель _____ Мандрикова Б.С.

Санкт-Петербург

2026

Цель работы.

Реализация метода нечеткой кластеризации с-средних с использованием природных временных рядов.

Задание.

Вариант 5. Станция Туле (THUL)

1. Скачать данные из источника [https://www.nmdb.eu/nest/] за периоды и станции, указанные по вариантам в Приложении 1 табл. п.1 и п.2. Использовать данные с одноминутным временным разрешением.

2. При наличии пропусков в данных заполнить их медианным значением (функция median в MatLab).

3. Сформировать матрицу входных данных data из преобразованных временных рядов в фиксированное число признаков (например, среднее значение, стандартное отклонение, амплитуда и др.).

4. Реализовать метод нечеткой кластеризации с-средних. В MatLab воспользоваться функцией: *[centers, U] = fcm(data, Nc, options)*, где data — входные данные (строки data соответствуют наблюдениям), Nc — количество кластеров, options — параметры метода.

5. Выполнить проверку работоспособности построенного метода кластеризации.

6. Сделать соответствующие выводы, оформить отчет по работе.

Выполнение работы.

Работа выполнялась на языке Python: numpy и pandas для обработки данных, matplotlib для визуализации. Исходный код см. в приложении А.

Данные станции Туле (THUL) скачаны с одноминутным разрешением. По таблице из условия лабораторной работы отобраны 45 спокойных суток (44 с удалением повтора) и 45 суток с магнитными бурями. Каждые сутки представляют собой временной ряд из 1440 отсчётов.

Пропуски в рядах заполняются медианным значением соответствующего ряда: ряд переиндексируетсяна полную сетку минут, недостающие отсчёты заменяются на медиану.

Каждый суточный ряд преобразовывается в вектор из 10 признаков: среднее значение, стандартное отклонение, амплитуда (max - min), медиана, минимум, максимум, 25-й и 75-й процентили, 1-й и 99-й процентили.

Так сформировывается матрица входных данных X. Перед кластеризацией признаки стандартизованы (нулевое среднее, единичная дисперсия), чтобы привести все признаки к одному масштабу.

Кластеризация выполнена нечетким методом с-средних при Nc = 2 кластерах, параметре размытости m = 2, точности 10^{-4}. Номер кластера для каждого наблюдения получен как argmax по строке результирующей матрицы принадлежностей U, (т. е. для каждого вектора выбирается кластер с наибольшим весом).

Нумерация кластеров произвольна, поэтому при сравнении с истинными метками кластеры сопоставлялись так, чтобы accuracy была максимальной. Если accuracy < 0.5, то кластеры перенумеруются.

Качество кластеризации оценивалось по матрице ошибок, accuracy, recall и precision для класса «магнитная буря», а также по индексу размытости FPC (fuzzy partition coefficient). FPC — это специальная метрика, вычисляющаяся как квадрат нормы Фробениуса результирующей матрицы принадлежностей. Она показывает «уверенность» модели в присвоении классов точкам (1 — уверенность большая, 1/c — уверенности нет, каждый класс для каждой точки равновероятен).

Результаты приведены в таблицах 1–2 и на рис. 1-3.

Таблица 1 — Метрики качества кластеризации

Метрика	Значение
accuracy	0.528
recall (спокойный)	0.773
precision (спокойный)	0.515
FPC	0.855

Таблица 2 — Матрица ошибок

Истинный класс \ предсказанный класс	спокойный	буря
спокойный	34	10
буря	32	13

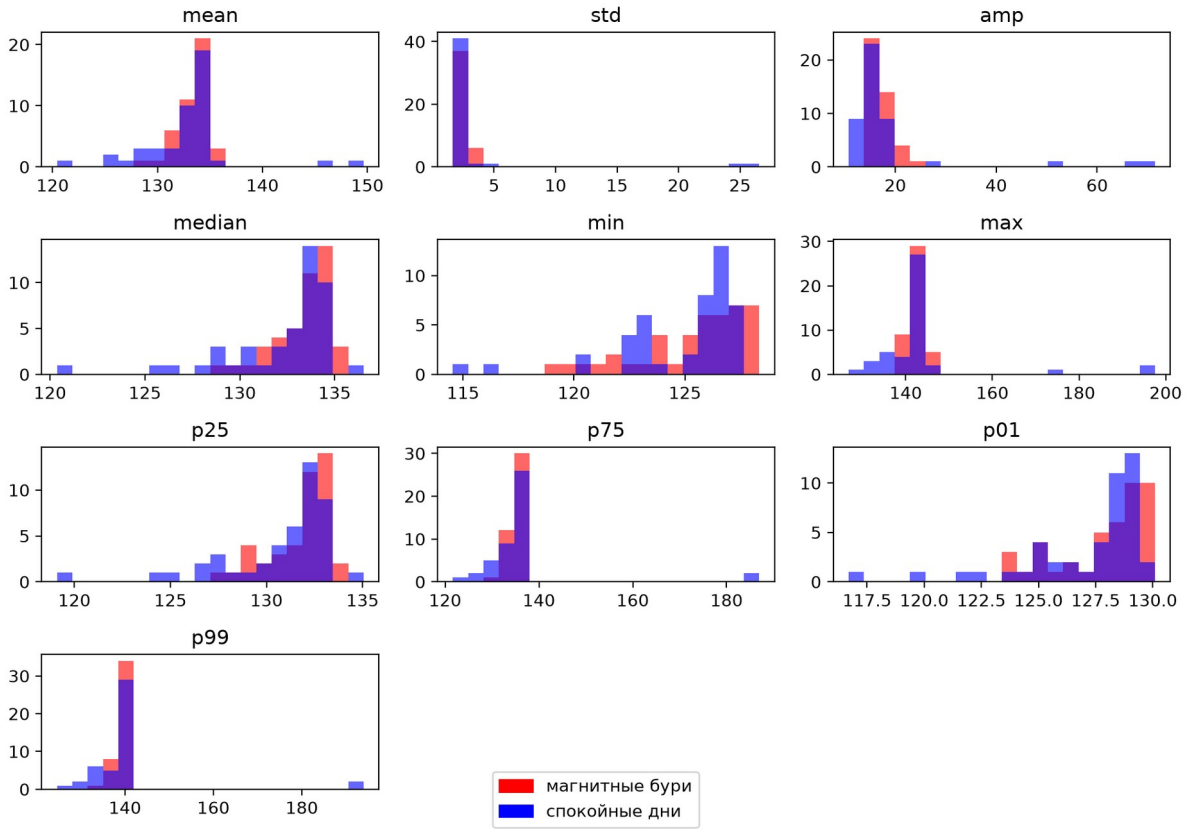


Рисунок 1 — Гистограммы распределений признаков (истинные)

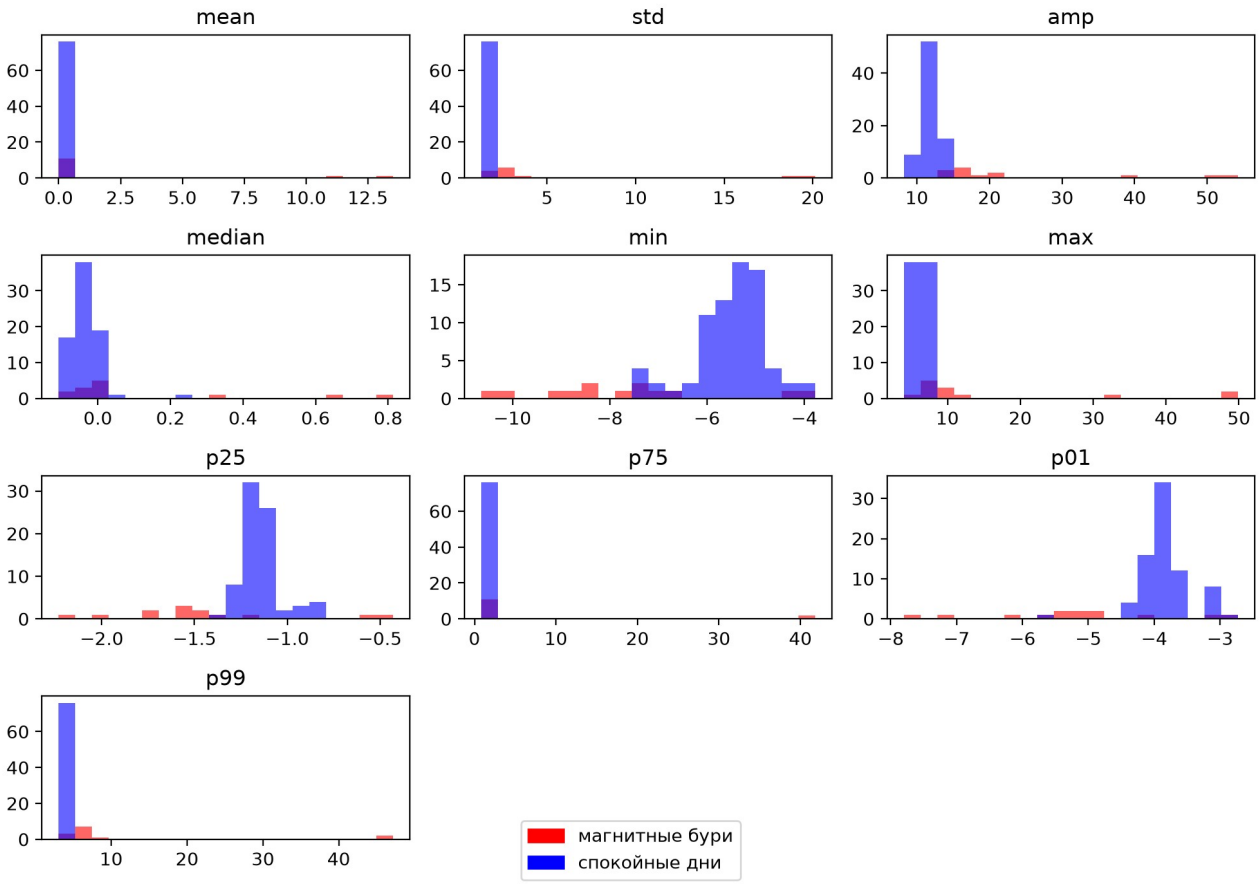


Рисунок 2 — Гистограммы распределений признаков (предсказанные)

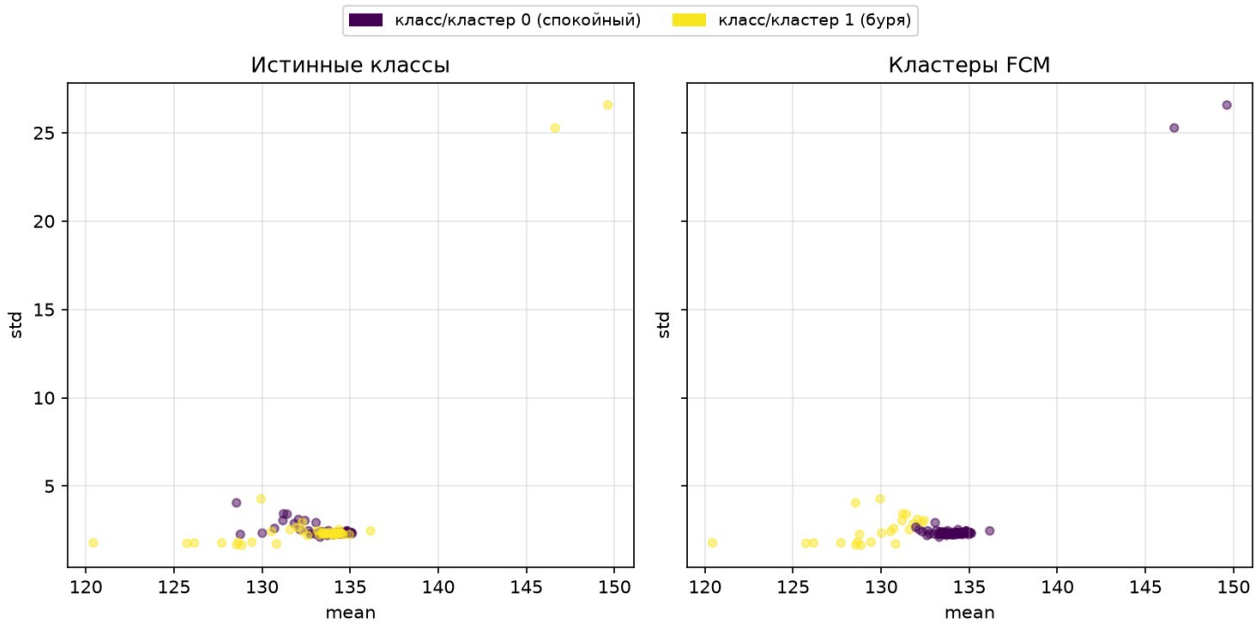


Рисунок 3 — Разделение на классы в разрезе среднее/отклонение (истинные классы и предсказанные моделью кластеры)

На рис. 1 и 2 mean, std, amp, median, min, max, p25, p75, p01, p99 — это среднее, отклонение, амплитуда, медиана, минимум, максимум, процентили соответственно.

Accuracy кластеризации составила 0.528 — лишь немного выше случайного угадывания. Из матрицы ошибок видно, что с-средних разделили 89 суток на два кластера неравного размера: большой и малый. Доли классов внутри кластеров близки к равным, то есть найденные кластеры практически не соответствуют истинным классам. Recall спокойных суток составил 0.773, однако precision спокойного класса равен лишь 0.515.

FPC = 0.855 указывает на достаточно чёткое разбиение (при полностью размытом разбиении FPC = 0.5), однако это разбиение почти однокластерное: основная масса суток сливается в один кластер, а второй кластер собирает лишь сутки с краю. На гистограммах (рис. 1) распределения признаков классов перекрываются по всем десяти признакам, визуально кластеры выделить невозможно. Алгоритм с-средних с задачей тоже не справился. Рис. 3 содержит сравнение выделенных кластеров с истинными классами в разрезе среднее/отклонение.

Выводы.

Реализован метод нечеткой кластеризации с-средних с использованием природных временных рядов.

Accuracy кластеризации составила 0.528, recall класса «спокойный» — 0.773, precision — 0.515, индекс размытости FPC — 0.855. Кластеры, найденные методом с-средних, не соответствуют размеченным данным. Рапределения выбранных признаков классов сильно перекрываются, поэтому алгоритм выделяет небольшой кластер аномальных суток на фоне неразделимой основной массы. Выбранные признаки недостаточно информативны для разделения классов.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

```
import matplotlib.pyplot as plt
from matplotlib.patches import Patch
import numpy as np
import pandas as pd
from pathlib import Path

#VALUE_COLUMN="RCORR_E"
VALUE_COLUMN="value"
DATA="data_raw"
SEED = 123
FEATURE_NAMES = [
    "mean", "std", "amp", "median",
    "min", "max",
    "p25", "p75", "p01", "p99",
]
M=2

def load_day(path: Path):
    df = pd.read_csv(path)
    df["timestamp"] = pd.to_datetime(df["timestamp"])
    df = df.set_index("timestamp")
    full = pd.date_range(df.index.min().normalize(), periods=1440, freq="1min")
    df = df.reindex(full)
    median = float(df[VALUE_COLUMN].median())
    df[VALUE_COLUMN] = df[VALUE_COLUMN].fillna(median)
    return df[VALUE_COLUMN].to_numpy(dtype=float)

def features(s):
    m = float(s.mean())
    sd = float(s.std())
    med = float(np.median(s))
    p25, p75 = np.percentile(s, [25, 75])
    p01, p99 = np.percentile(s, [1, 99])
    return np.array([
        m, sd, s.max() - s.min(), med,
        s.min(), s.max(),
        p25, p75, p01, p99,
    ])

def load_all():
    X, y = [], []
    for cls in (0, 1):
        for path in sorted((Path(DATA) / f"class{cls}").glob("*.csv")):
            series = load_day(path)
            X.append(features(series))
            y.append(cls)
    return np.array(X), np.array(y)

def match_to_pred_labels(y_true, y_pred):
    acc1 = float(np.mean(y_pred == y_true))
    acc2 = float(np.mean(y_pred == 1 - y_true))
    if acc2 > acc1:
        return acc2, 1 - y_pred
    return acc1, y_pred

def confusion(y_true, y_pred):
    return np.array([[np.sum((y_true == 0) & (y_pred == 0)),
                      np.sum((y_true == 0) & (y_pred == 1))],
```

```
                    [np.sum((y_true == 1) & (y_pred == 0)),
                     np.sum((y_true == 1) & (y_pred == 1))]])

def fcm(X, n_clusters, m, max_iter, tol, seed):
    n = len(X)
    U = np.random.default_rng(seed).random((n, n_clusters))
    U = U / U.sum(axis=1, keepdims=True)

    for _ in range(max_iter):
        w = U ** m
        centers = (w.T @ X) / w.sum(axis=0)[:, None]

        d2 = ((X[:, None, :] - centers[None, :, :]) ** 2).sum(axis=2)
        d2 = np.maximum(d2, 1e-12)
        U_new = 1.0 / d2 ** (1.0 / (m - 1))
        U_new = U_new / U_new.sum(axis=1, keepdims=True)

        if np.abs(U_new - U).max() < tol:
            U = U_new
            break
        U = U_new

    w = U ** m
    centers = (w.T @ X) / w.sum(axis=0)[:, None]
    return centers, U

def draw_gist(ys, filename):
    cols = {0: "#FF0000", 1: "#0000FF"}
    fig, axes = plt.subplots(4, 3, figsize=(10, 7))
    for i, ax in enumerate(axes.flat):
        if i >= len(FEATURE_NAMES):
            ax.axis("off")
            continue

        f0, f1 = X[ys == 0, i], X[ys == 1, i]
        bins = np.histogram_bin_edges(np.concatenate([f0, f1]), bins=20)
        ax.hist(f0, bins=bins, alpha=0.6, color=cols[0])
        ax.hist(f1, bins=bins, alpha=0.6, color=cols[1])
        ax.set_title(FEATURE_NAMES[i])

    fig.legend(handles=[Patch(color=cols[0], label="магнитные бури"),
                        Patch(color=cols[1], label="спокойные дни")],
            loc="lower center")
    fig.tight_layout()
    fig.savefig(filename, dpi=200)
    plt.close(fig)

X, y = load_all()

std = X.std(axis=0)
std[std == 0] = 1.0
Xn = (X - X.mean(axis=0)) / std

centers, U = fcm(Xn, 2, M, 100, 1e-4, SEED)
fpc = float(np.mean(np.sum(U ** 2, axis=1)))

y_pred = U.argmax(axis=1)
acc, y_pred = match_to_pred_labels(y, y_pred)
cm = confusion(y, y_pred)

print(f"accuracy: {acc:.3f}")
print("confusion matrix:")
print(cm)
```

```
tp, fn, fp, tn = cm[0, 0], cm[0, 1], cm[1, 0], cm[1, 1]
recall = tp / (tp + fn)
precision = tp / (tp + fp)
print(f"recall (спокойные): {recall:.3f}")
print(f"precision (спокойные): {precision:.3f}")
print(f"FPC: {fpc:.3f}")

fig, axes = plt.subplots(1, 2, figsize=(10, 5), sharex=True, sharey=True)
for ax, colors, title in [
    (axes[0], y, "Истинные классы"),
    (axes[1], y_pred, "Кластеры FCM"),
]:
    sc = ax.scatter(X[:, 0], X[:, 1], c=colors, cmap="viridis", s=20, alpha=0.5)
    ax.set_xlabel(FEATURE_NAMES[0])
    ax.set_ylabel(FEATURE_NAMES[1])
    ax.set_title(title)
    ax.grid(alpha=0.3)

fig.legend(handles=[Patch(color=plt.cm.viridis(0), label="класс/кластер 0
(спокойный)"),
                    Patch(color=plt.cm.viridis(0.99), label="класс/кластер 1
(буря)")],
           loc="upper center", ncol=2, fontsize=9)
fig.tight_layout(rect=(0, 0, 1, 0.94))
fig.savefig("img/fcm_clusters.png", dpi=150)

draw_gist(y, "img/feat_hists.png")
draw_gist(y_pred, "img/feat_hists_pred.png")
```